A Procedural-Track Self-Driving Car Simulator Using Trajectory-Based Actor-Critic Reinforcement Learning

Aashish M
EP23BT005
Project as part of the course:
EE608T: Neural Networks and Deep Learning

***Abstract*—This report presents a complete 2D autonomous driving project implemented with PyTorch and Pygame, in which an agent learns to drive on closed-loop tracks using actor-critic reinforcement learning. The system combines a procedurally generated road environment, simplified vehicle dynamics, ray-cast distance sensors, continuous control actions, and a trajectory-based policy/value optimization pipeline using Generalized Advantage Estimation (GAE). The report documents the full implementation at the level of executable code: track generation through periodic B-spline interpolation, observation and reward engineering, actor and critic network architectures, optimization details, checkpointing and visualization utilities, and the runtime demo configuration. In addition to describing the design, the report records the exact hyperparameters used in the codebase and analyzes practical implementation choices that make the system trainable and visually inspectable.**

***Index Terms*—reinforcement learning, actor-critic, autonomous driving, procedural track generation, Pygame, PyTorch, Generalized Advantage Estimation**

I. Project Repository

The full implementation corresponding to this report is available in a public GitHub repository:

https://github.com/AashishMK10/self_driving_ac

The repository contains the environment, model definitions, training pipeline, visualization utilities, and checkpoints referenced throughout this report.

II. Introduction

Autonomous driving provides a useful benchmark for continuous-control reinforcement learning because it requires perception, control, sequential decision making, and robustness to varied geometry. This project addresses the task in a simplified 2D top-down setting. A car must remain on a closed road while maximizing forward progress. The road can either be a fixed oval or a newly generated procedural track at every episode, forcing the policy to generalize rather than memorize a single layout.

The implementation is deliberately compact and educational. It avoids external RL frameworks, defines the environment from first principles, and builds the actor-critic agent directly in PyTorch. The resulting system is fully runnable and supports both training and interactive demo playback in a Pygame window.

The main design choices are motivated by computational and methodological tradeoffs rather than by convenience alone. Actor-critic learning is a natural fit because steering and throttle are continuous controls; unlike discrete-action Q-learning, it does not require coarse action discretization that would either limit maneuver precision or enlarge the action table combinatorially. The 2D top-down simulator sacrifices photorealism in exchange for rapid iteration, transparent debugging, and direct access to geometric state variables, which is appropriate for studying the reinforcement learning pipeline itself. Procedural track generation similarly trades benchmark determinism for broader state coverage, making the learned policy confront varied curvature, corridor width, and recovery scenarios instead of over-specializing to a single hand-designed course.

III. Related Work

Actor-critic methods are a standard foundation for continuous-control reinforcement learning. Advantage Actor-Critic (A2C) showed that policy-gradient learning can be stabilized by coupling a stochastic policy with a learned value baseline, providing a conceptual template for the trajectory-based updates used here [1]. Proximal Policy Optimization (PPO) later improved robustness by constraining policy updates with a clipped surrogate objective, often making it a stronger default for on-policy control than plain actor-critic [2]. Soft Actor-Critic (SAC) and Twin Delayed DDPG (TD3) represent influential off-policy alternatives for continuous actions, typically offering higher sample efficiency through replay-based learning, but at the cost of additional algorithmic complexity that this educational implementation deliberately avoids [3], [4]. With respect to driving environments, compact simulators such as TORCS have long been used for autonomous racing research because they expose the control problem without the full burden of real-world perception [5]. More broadly, procedural content generation has been repeatedly used in reinforcement learning to improve robustness and reduce memorization of fixed layouts, which directly motivates the random closed-loop track design adopted in this project [6].

IV. Project Objectives

The project was designed around the following objectives:

- Learn continuous steering and throttle control from low-dimensional observations.
- Train on procedurally generated tracks to reduce overfitting to one geometry.
- Preserve an interpretable pipeline in which the environment, reward function, networks, and update rule are explicit.
- Provide real-time visualization for debugging, qualitative evaluation, and demonstration.

V. System Overview

The project consists of five tightly coupled subsystems:

1) **Environment**: a Gym-style driving environment managing reset, step, reward, termination, and rendering.
2) **Track model**: a closed centerline with inner and outer boundaries, either fixed or procedurally generated.
3) **Vehicle model**: a simplified kinematic car with heading, speed, and angular velocity.
4) **Perception**: a fan of ray-cast distance sensors returning normalized boundary distances.
5) **Learning agent**: an actor-critic policy/value pair updated episode-by-episode using GAE.

At runtime, the environment produces an observation vector from sensors and vehicle-track geometry. The actor samples a continuous action, the car state advances, the environment computes reward and termination, and the trajectory is stored. After each episode, the collected trajectory is used in a batched update of the actor and critic.

VI. Environment Design

A. State Representation

The observation vector has dimension

$$d_{\text{obs}} = N_{\text{sensors}} + 4, \tag{1}$$

where $N_{\text{sensors}} = 9$ in the current configuration. The observation is

$$s_t = [r_1, \ldots, r_9, \hat{v}, \hat{\theta}_{rel}, \hat{e}_{lat}, \hat{\omega}], \tag{2}$$

with:

- $r_i \in [0, 1]$: normalized ray distances,
- $\hat{v} = v/v_{\max}$: normalized speed,
- $\hat{\theta}_{rel} \in [-1, 1]$: heading difference from local road tangent divided by π,
- $\hat{e}_{lat} \in [-1, 1]$: lateral offset normalized by half the road width,
- $\hat{\omega} \in [-1, 1]$: angular velocity scaled and clipped by 0.5.

Using a low-dimensional state is a deliberate choice: it removes the need for image encoders and focuses learning on control-relevant geometry. The additional four non-sensor terms complement the ray fan by explicitly exposing quantities that are expensive for a shallow MLP to infer indirectly, namely progress alignment, centerline deviation, and rotational motion.

B. Action Space

The agent outputs a two-dimensional continuous action:

$$a_t = [u_t^{steer}, u_t^{throttle}]. \tag{3}$$

The environment interprets these as:

- steering in $[-1, 1]$,
- throttle in $[0, 1]$.

The raw actor output is Gaussian and unconstrained before post-processing. Steering is clamped directly, while throttle is mapped from $[-1, 1]$ into $[0, 1]$ by

$$u^{throttle} = \text{clip}\left(\frac{a_2 + 1}{2}, 0, 1\right). \tag{4}$$

This separation reflects the asymmetry of the task. Steering benefits from signed control around zero, whereas throttle is intentionally restricted to nonnegative values to keep the vehicle model simple and to prevent the policy from exploiting reverse motion as an artificial recovery strategy.

C. Episode Lifecycle

At each reset, the environment optionally regenerates the track, places the car at the first centerline point, aligns the car heading with the local tangent, and clears reward and step counters. An episode terminates under either of two conditions:

- the car leaves the road, treated as a collision,
- the number of steps reaches the configured maximum of 3000 (the number was later changed to 300 for brevity's sake).

VII. Procedural Track Generation

A. Fixed Oval Baseline

When procedural generation is disabled, the centerline is a discretized ellipse:

$$x(\phi) = c_x + r_x \cos\phi, \tag{5}$$

$$y(\phi) = c_y + r_y \sin\phi, \tag{6}$$

sampled at uniformly spaced angles. This mode is useful for debugging and reproducibility.

B. Random Closed-Loop Track Construction

The default training mode uses random tracks. The generator constructs a smooth closed-loop centerline in four stages:

1) Evenly spaced base waypoint angles are created around the screen center.
2) Each angle receives bounded random angular jitter to avoid degenerate symmetry.
3) Each waypoint is assigned a random radius between a minimum and maximum bound.
4) A periodic cubic B-spline interpolates the closed waypoint set to obtain a smooth centerline.

The track generator is intentionally structured this way to balance diversity with solvability. The bounded waypoint jitter

and radius limits produce nontrivial turns while avoiding self-intersections or extremely narrow radii that would dominate the learning signal with unavoidable collisions.

Formally, for K waypoints,

$$\phi_i = \frac{2\pi i}{K} + \Delta\phi_i, \tag{7}$$

where $\Delta\phi_i$ is uniform angular jitter, and

$$\rho_i \sim \mathcal{U}(\rho_{min}, \rho_{max}). \tag{8}$$

Waypoint coordinates are then

$$x_i = c_x + \rho_i \cos\phi_i, \tag{9}$$

$$y_i = c_y + \rho_i \sin\phi_i. \tag{10}$$

The first waypoint is appended to the end of the sequence to explicitly close the loop, and SciPy's `splprep`/`splev` with `per=True`, smoothing factor $s = 0$, and cubic order $k = 3$ generate a periodic interpolating spline. The implementation samples 300 points from the spline to obtain the final centerline. Choosing a periodic cubic spline preserves smooth heading transitions around the entire loop, which is important because sharp piecewise-linear corners would create unrealistic steering discontinuities. Sampling 300 points is a practical resolution tradeoff: dense enough for stable tangent and progress estimates, but still cheap for repeated nearest-point and rendering operations.

C. Boundary Construction

Once the centerline is available, the road boundaries are created by offsetting each centerline point along its local normal. For centerline point $\mathbf{p}_i$, a local tangent is approximated using neighboring points:

$$\mathbf{t}_i = \frac{\mathbf{p}_{i+1} - \mathbf{p}_{i-1}}{\|\mathbf{p}_{i+1} - \mathbf{p}_{i-1}\| + \epsilon}. \tag{11}$$

The left-pointing normal is

$$\mathbf{n}_i = [-t_{i,y}, t_{i,x}]. \tag{12}$$

Using half-width $w/2$, the inner and outer boundaries are:

$$\mathbf{b}_i^{inner} = \mathbf{p}_i + \mathbf{n}_i \frac{w}{2}, \tag{13}$$

$$\mathbf{b}_i^{outer} = \mathbf{p}_i - \mathbf{n}_i \frac{w}{2}. \tag{14}$$

These boundaries are converted into line segments used by the sensor ray-caster.

D. Progress and Lateral Offset

The centerline is also used to estimate:

- cumulative arc-length for progress measurement,
- local tangent heading,
- signed lateral offset from the centerline.

Progress is stored by accumulating segment lengths over the centerline. The road heading at the closest centerline point is computed from the local tangent. Lateral offset is the signed projection of the car position onto the local normal.

E. Road Membership Test

The road-membership predicate is intentionally simple: a point is considered on-road if its minimum Euclidean distance to the centerline is less than half the track width. The car is considered on-road only if all four rectangle corners satisfy this test. This is computationally cheap and works well with moderate track curvature.

VIII. Vehicle Dynamics

The car model is a lightweight 2D kinematic approximation with state

$$\mathbf{x}_t = [x_t, y_t, \theta_t, v_t, \omega_t]. \tag{15}$$

At each time step, steering and throttle are applied as:

$$\omega_t = v_t\, u_t^{steer}\, k_{turn}, \tag{16}$$

$$\theta_{t+1} = \theta_t + \omega_t \Delta t, \tag{17}$$

$$v_{t+1} = \text{clip}\left(v_t + (u_t^{throttle} a - f v_t)\Delta t, 0, v_{max}\right), \tag{18}$$

and the position update is

$$x_{t+1} = x_t + v_{t+1} \cos\theta_{t+1} \Delta t, \tag{19}$$

$$y_{t+1} = y_t + v_{t+1} \sin\theta_{t+1} \Delta t. \tag{20}$$

This model captures three desirable effects:

- turning becomes harder at low speed because angular velocity scales with speed,
- friction damps velocity naturally,
- the dynamics remain simple enough for fast simulation and visual intuition.

IX. Ray-Cast Sensor Model

The car uses 9 ray sensors spread symmetrically across $180°$ around the current heading. If the heading is θ, sensor directions are

$$\theta_i = \theta + \delta_i, \tag{21}$$

where the offsets δ_i are linearly spaced from $-90°$ to $+90°$. This configuration intentionally favors interpretability over maximal sensing richness. Nine rays are sufficient to expose near-field road geometry and upcoming turns without making the observation vector unnecessarily redundant, while the $180°$ spread gives the agent broad forward-side awareness yet avoids spending sensors on regions directly behind the car that are less relevant under a no-reverse driving model.

For each ray, the implementation tests intersection against every boundary segment in a vectorized NumPy computation. Let a segment start at $\mathbf{s}$ and end at $\mathbf{e}$, with segment vector $\mathbf{v}_2 = \mathbf{e} - \mathbf{s}$. Let $\mathbf{o}$ be the ray origin and $\mathbf{d}$ the ray direction. The algorithm computes parameters corresponding to distance along the ray and normalized position along the segment. Hits are valid when the intersection lies in front of the ray and within the segment bounds. The minimum valid hit distance is retained and clipped to the maximum sensor range. Distances are normalized to $[0, 1]$.

This implementation avoids Python loops over track segments. Only the outer loop over sensors remains, while each sensor tests all segments simultaneously.

X. Reward Function

The reward is a weighted sum of progress, speed encouragement, centering, smoothness, and collision terms:

$$r_t = r_t^{prog} + r_t^{speed} + r_t^{center} + r_t^{steer} + r_t^{collision} + r_t^{alive}. \quad (22)$$

The individual components are:

$$r_t^{prog} = \Delta i_t \cdot w_{prog}, \quad (23)$$

$$r_t^{speed} = w_{speed} \frac{v_t}{v_{max}}, \quad (24)$$

$$r_t^{center} = -w_{center} \frac{|e_{lat}|}{w/2}, \quad (25)$$

$$r_t^{steer} = -w_{steer} |u_t^{steer} - u_{t-1}^{steer}|, \quad (26)$$

$$r_t^{collision} = \begin{cases} w_{collision}, & \text{if off-road} \\ 0, & \text{otherwise} \end{cases} \quad (27)$$

with $r_t^{alive} = w_{alive}$.

In the current implementation:

- progress uses centerline index advancement rather than exact arc-length displacement,
- wrap-around artifacts are handled by capping forward jumps larger than half the track length and interpreting them as backward movement,
- the alive term is configured to zero, so it does not currently shape behavior.

The reward weights are chosen to prioritize forward movement while discouraging visibly unstable driving. In particular, the speed bonus is kept smaller than the progress term so that the policy cannot maximize reward by accelerating without regard to track following, while the center and steering-change penalties regularize the solution toward lane-keeping and smoother control rather than oscillatory edge-tracking.

XI. Learning Algorithm

A. Actor-Critic Formulation

The project uses a continuous-action actor-critic architecture. The actor represents a stochastic policy $\pi_\theta(a|s)$, and the critic approximates the state-value function $V_\psi(s)$. During data collection, the actor samples actions and the critic estimates the current state's value. After a full episode is collected, both networks are updated.

B. Generalized Advantage Estimation

Advantages are computed using GAE. Given reward r_t, value estimate $V(s_t)$, and terminal indicator d_t, the temporal-difference residual is

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_t) - V(s_t). \quad (28)$$

The advantage is then recursively accumulated as

$$A_t = \delta_t + \gamma\lambda(1 - d_t)A_{t+1}. \quad (29)$$

The implementation computes this recursion backward over the episode on the CPU, then forms returns as

$$R_t = A_t + V(s_t). \quad (30)$$

Advantages are normalized to zero mean and unit variance for stability when the trajectory length exceeds one sample. The choice $\lambda = 0.95$ is a standard bias-variance compromise: lower values would produce noisier but more myopic advantages, whereas values too close to 1 would make updates sensitive to long-horizon return variance on randomly generated tracks. Advantage normalization further reduces scale drift across episodes of different quality and length.

C. Loss Functions

The critic is trained with mean squared error:

$$\mathcal{L}_{critic} = \frac{1}{T} \sum_{t=1}^{T} \left(V_\psi(s_t) - R_t\right)^2. \quad (31)$$

The actor uses the episodic policy-gradient objective with entropy regularization:

$$\mathcal{L}_{actor} = -\frac{1}{T} \sum_{t=1}^{T} \log \pi_\theta(a_t|s_t) A_t - \beta \frac{1}{T} \sum_{t=1}^{T} \mathcal{H}\left[\pi_\theta(\cdot|s_t)\right]. \quad (32)$$

Here, β is the entropy coefficient. Gradient clipping is applied to both networks when the configured threshold is positive. The entropy coefficient of 0.02 is large enough to discourage premature collapse to a nearly deterministic policy during early exploration, but small enough that the agent can still sharpen its control once consistent progress is discovered. Gradient clipping at norm 0.5 plays a complementary role by limiting destabilizing updates when a long successful or catastrophic episode produces unusually large return targets.

XII. Neural Network Architectures

A. Actor Network

The actor is a multilayer perceptron with:

- input dimension 13,
- two fully connected hidden layers of width 128,
- ReLU activations,
- a linear mean head followed by $\tanh$,
- a state-independent learnable log standard deviation vector.

The resulting policy is a diagonal Gaussian:

$$\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \mathrm{diag}(\sigma^2)). \quad (33)$$

The log standard deviation is clamped elementwise to the interval $[-2.0, 0.5]$ before exponentiation. This keeps exploration bounded and improves numerical stability. The two-layer, width-128 MLP is a practical middle ground for this state size. It is expressive enough to model nonlinear interactions between sensor geometry and vehicle state, while remaining small enough that training remains fast and failures can still be diagnosed from logged behavior rather than hidden behind a heavily overparameterized model.

B. Critic Network

The critic is also an MLP with:

- input dimension 13,
- two fully connected hidden layers of width 128,
- ReLU activations,
- a scalar output representing $V(s)$.

Using the same hidden width as the actor keeps representation capacity roughly matched between policy and value learning. That symmetry reduces one common failure mode in small actor-critic systems, namely a weak critic producing high-variance advantages that the actor then overfits.

C. Implementation Note

The configuration file exposes a parameter `num_hidden_layers`, but the current actor and critic implementations instantiate exactly two hidden layers. Thus, the report documents the realized model rather than the configurable intention. If need be, the MLP can be extended to contain more hidden layers and use the config. file variable, but two hidden layers is deemed sufficient for this scale.

XIII. Training Procedure

Training is episodic and trajectory-based:

1) Reset the environment, optionally generating a new track.
2) Roll out one full episode using the stochastic actor.
3) Store observations, raw actions, rewards, done flags, and critic values.
4) Bootstrap the final value from the last observation.
5) Compute GAE advantages and returns.
6) Perform one batch critic update and one batch actor update.
7) Log metrics, save checkpoints periodically, and update the best model if reward improves.

This design differs from fully online step-by-step actor-critic updates. The implemented approach uses one full-trajectory update per episode, which improves batching efficiency and pairs naturally with GAE. One update per episode is also a deliberate engineering simplification. It removes bookkeeping around minibatching and multiple epochs over stale data, but the tradeoff is lower sample efficiency and slower correction when a single poor trajectory dominates the update.

XIV. Software Architecture

The codebase is organized into modules with clear responsibilities:

- `main.py`: command-line entry point for training and demo modes.
- `config.py`: centralized hyperparameters.
- `env/track.py`: fixed and procedural track generation.
- `env/car.py`: kinematic vehicle model.
- `env/sensors.py`: vectorized ray-cast perception.
- `env/environment.py`: observation, reward, reset, step, and rendering.
- `models/actor.py`: Gaussian policy network.
- `models/critic.py`: state-value network.
- `rl/agent.py`: action selection, GAE, optimization, checkpoint I/O.
- `rl/train.py`: training loop and deterministic demo loop.
- `utils/logger.py`: episode-level metric storage.
- `utils/visualization.py`: reward and loss plotting.

XV. Implementation Hyperparameters

Table I records the exact values present in the project configuration.

XVI. Rendering and Visualization

The project uses Pygame for real-time visualization. The renderer displays:

- the outer road polygon,
- the inner polygonal hole,
- a dashed centerline,
- color-coded sensor rays,
- the car body and forward heading line,
- a HUD with episode, step count, speed, and reward.

For offline analysis, Matplotlib is used with a non-interactive backend to save:

- `reward_curve.png`,
- `loss_curves.png`,
- `track_preview.png` for track samples.

XVII. Experimental Artifacts

The repository already contains trained checkpoints for the actor and critic, as well as generated plots. The demo mode loads the best checkpoint if available; otherwise it falls back to the latest checkpoint. In a verified local run of the shipped demo, the deterministic policy completed an episode of 3000 steps with a total reward of approximately 4856.3 before episode timeout.

Figure 1 illustrates procedural track diversity. Figures 2 and 3 show the saved training diagnostics from the project workspace.

The reward curve indicates a plausible staged learning process rather than an immediate jump to competent driving. Early episodes remain near zero reward, which is consistent with a randomly initialized policy that either leaves the track quickly or makes only negligible progress before collision. The most substantial improvement occurs roughly between episodes 200 and 400, where return rises markedly; this phase is consistent with the policy discovering a usable coordination between heading alignment, throttle application, and centerline recovery. The final demonstrated reward of approximately 4856.3 over the full 3000-step horizon suggests that the trained controller does not merely survive briefly, but is capable of sustaining forward motion for an entire episode on the evaluated track.

The loss curves should be interpreted together with the return trend. In actor-critic training, perfectly monotone losses

TABLE I
PROJECT HYPERPARAMETERS FROM `CONFIG.PY`

Category	Parameter	Value
Reinforcement learning	Actor learning rate	3×10^{-4}
	Critic learning rate	1×10^{-3}
	Discount factor γ	0.99
	GAE λ	0.95
	Entropy coefficient	0.02
	Gradient clipping norm	0.5
Network	Hidden width	128
	Configured hidden-layer count	2
Environment and track	Number of sensors	9
	Sensor length	150.0 px
	Sensor spread	180.0°
	Track width	80.0 px
	Maximum episode steps	300
	Random track enabled	True
	Minimum waypoint radius	150.0 px
	Maximum waypoint radius	280.0 px
Random track detail	Number of random waypoints	12
	Interpolated centerline points	300
Car physics	Car length	20.0 px
	Car width	10.0 px
	Maximum speed	8.0 px/step
	Acceleration coefficient	0.4
	Friction coefficient	0.05
	Turn-rate coefficient	0.04
	Simulation time step Δt	1.0
Training and runtime	Number of episodes	5000
	Checkpoint interval	100
	Log interval	10
	Render during training	True
	Training FPS cap	0 (uncapped)
	Demo FPS cap	60
Reward shaping	Progress weight	1.0
	Speed bonus weight	0.1
	Center penalty weight	0.3
	Steering-change penalty weight	0.05
	Collision penalty	-10.0
	Alive cost	0.0
Display	Screen size	1000×700

are neither expected nor desirable because the targets themselves move as the policy improves. What matters is whether the critic loss remains bounded and whether the actor loss avoids persistent high-amplitude oscillation or divergence. The saved diagnostics are consistent with a training process that becomes progressively more stable after the initial exploration regime: once reward improves, the critic appears to provide sufficiently coherent value estimates for GAE to produce usable policy gradients, and the actor no longer behaves as though it is relearning from scratch each episode.

Qualitatively, a well-trained policy in this environment tends to maintain moderate forward speed, begin steering before reaching the road boundary, and make small continuous corrections instead of alternating between extreme left and right commands. Successful rollouts usually show the car staying near the centerline on straights and smoothly shifting laterally before entering bends. The most common failure modes are late turning on tighter curves, overcorrection after approaching an edge, and occasional oscillatory steering that converts a recoverable deviation into an off-road termination. These behaviors are consistent with the limits of sparse geometric sensing and single-trajectory updates: the agent learns competent lane following, but not a fully robust recovery strategy under every randomized geometry.

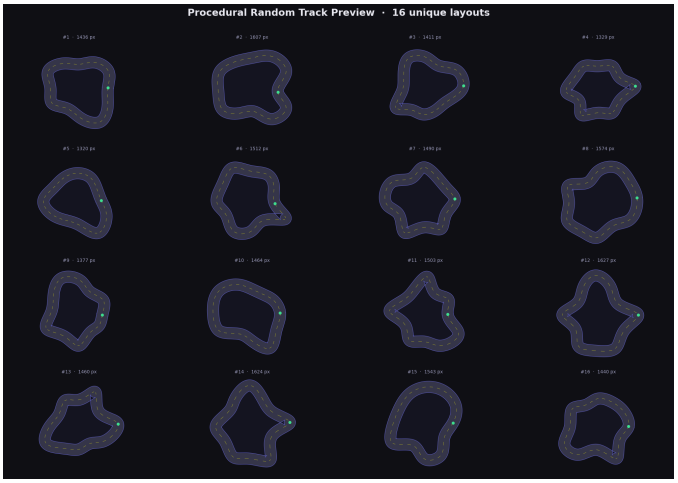

Fig. 1. Preview of procedurally generated closed-loop tracks.

Fig. 2. Saved reward curve from training.

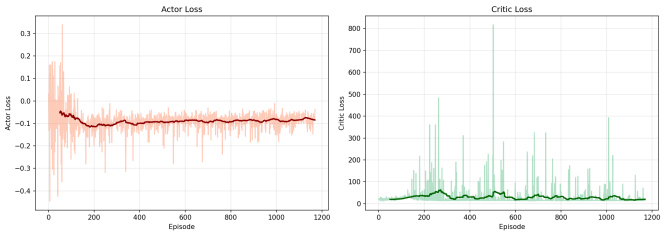

Fig. 3. Saved actor and critic loss curves from training.

XVIII. Strengths of the Implementation

The system has several strong engineering qualities:

- **Generalization pressure**: random track generation forces the policy to handle varying geometry.
- **Low-dimensional perception**: ray sensors provide a compact observation space without image processing overhead.
- **Efficient updates**: trajectory batching reduces Python overhead and uses tensor operations efficiently.
- **Interpretability**: every state component, reward term, and dynamic update is easy to inspect.
- **Reproducible software structure**: the code cleanly separates environment, models, RL logic, and utilities.

XIX. Limitations

The project also has limitations that should be acknowledged in a professional report:

- The road-membership test is centerline-distance based rather than polygon exact, which may be approximate on sharp curvature.
- The car model is kinematic and omits lateral tire slip, braking, and reverse motion.
- The actor uses a state-independent standard deviation, limiting adaptivity of exploration. The same exploration scale is therefore applied in both easy straight segments and difficult high-curvature regions, even though those situations demand different uncertainty levels.
- Training uses one update per episode, which is simple but less sample-efficient than more advanced on-policy optimizers.

XX. Potential Extensions

Several technically meaningful extensions follow naturally:

- switch from centerline-distance collision checks to exact polygon containment or signed distance fields,
- add richer dynamics such as braking, reverse gear, slip, or curvature-aware steering,
- condition policy variance on the state,
- compare the current approach against PPO, SAC, or TD3,
- introduce curriculum learning over track complexity,
- log evaluation metrics on held-out seeds for more rigorous generalization analysis.

XXI. Conclusion

This project demonstrates a complete reinforcement learning pipeline for 2D self-driving in a procedurally generated environment. The implementation is educational but nontrivial: it includes smooth random track synthesis with periodic splines, interpretable geometric observations, continuous control, trajectory-based actor-critic optimization with GAE, real-time visualization, and saved training diagnostics. As a course project, it is strong because it integrates environment design, algorithm implementation, reward engineering, simulation, and empirical results into a single coherent system. The project is also a practical foundation for future work on more advanced vehicle dynamics, stronger RL algorithms, and more rigorous evaluation protocols.

Appendix: Reproduction Commands

The following commands reproduce the main project workflows:

- Training with rendering: `python main.py -mode train`
- Training without rendering: `python main.py -mode train -no-render`

- Resume training: `python main.py -mode train -resume`
- Deterministic demo: `python main.py -mode demo`
- Track preview generation: `python track_preview.py`

REFERENCES

[1] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Machine Learning (ICML)*, 2016, pp. 1928–1937.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Machine Learning (ICML)*, 2018, pp. 1861–1870.

[4] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Machine Learning (ICML)*, 2018, pp. 1587–1596.

[5] B. Wymann, C. Dimitrakakis, A. Sumner, E. Espié, and C. Guionneau, "TORCS: The open racing car simulator," 2000. [Online]. Available: http://torcs.sourceforge.net

[6] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman, "Leveraging procedural generation to benchmark reinforcement learning," in *Proc. Int. Conf. Machine Learning (ICML)*, 2020, pp. 2048–2056.